BROZER: A PRIVACY BOUNDARY FOR BROWSER AGENTS WITH ON-DEVICE VISUAL PERCEPTION

PROJECT REPORT

Submitted by

PADMANABHAN SURESH BABU (714025104188)

MITHUNEESH K (714025104144)

NISHANTH KR (714025104173)

NITISH RG (714025104183)

In partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

SRI SHAKTHI

INSTITUTE OF ENGINEERINGAND TECHNOLOGY,

An Autonomous Institution

APRIL 2026

BONAFIDE CERTIFICATE

BONAFIDE CERTIFICATE

Certified that this project report titled "**BROZER: A PRIVACY BOUNDARY FOR BROWSER AGENTS WITH ON-DEVICE VISUAL PERCEPTION**" is the bonafide work of **PADMANABHAN SURESH BABU (714025104188), MITHUNEESH K (714025104144), NISHANTH KR (714025104173), and NITISH RG (714025104183)**, who carried out the work under my supervision.

SIGNATURE
Mr. B.SANJAY KRISHNA

SUPERVISOR
Assistant Professor,
Department of CSE,
Sri Shakthi Institute of
Engineering and Technology,
Coimbatore – 641062.

SIGNATURE
Dr.K.E.Kannammal

HEAD OF THE DEPARTMENT
Professor and Head,
Department of CSE,
Sri Shakthi Institute of
Engineering and Technology,
Coimbatore – 641062.

Submitted for the project work viva-voce Examination held on.....

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

ACKNOWLEDGEMENT

First and foremost, I would like to thank God Almighty for giving me the strength. Without His blessings this achievement would not have been possible.

We express our deepest gratitude to our **Chairman Dr. S. Thangavelu** for his continuous encouragement and support throughout our course of study.

We are thankful to our **Secretary Mr. T. Dheepan** for his unwavering support during the entire course of this project work.

We are also thankful to our **Joint Secretary Mr. T. Sheelan** for his support during the entire course of this project work.

We are highly indebted to **Principal Dr. N. K. Sakthivel** for his support during the tenure of the project.

We are deeply indebted to our **Head of the Department**, Computer Science and Engineering, **Dr. K. E. Kannammal** for providing us with the necessary facilities.

It's a great pleasure to thank our **Project Guide Mr. B. SANJAY KRISHNA , AP/CSE** for his valuable technical suggestions and continuous guidance throughout this project work.

We are also thankful to our **Project Coordinator** for providing us with the necessary facilities and encouragement.

We solemnly extend our thanks to all the **teachers and non-teaching staff** of our department, **family**, and **friends** for their valuable support.

PADMANABHAN SURESH BABU
MITHUNEESH K
NISHANTH KR
NITISH RG

TABLE OF CONTENTS

TABLE OFCONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	
	LIST OF ABBREVATIONS	
	LIST OF FIGURES	
1	**INTRODUCTION**	**1**
2	**LITERATURE SURVEY**	**4**
3	**TECHNOLOGY STACK**	**14**
4	**IMPLEMENTATION OF BROZER PRIVACY BOUNDARY SYSTEM**	**18**
	4.1 EXISTING SYSTEM	
	4.2 DRAWBACKS	
	4.3 PROPOSED SYSTEM	
	4.4 MERITS	
	4.5 ALGORITHM	
	4.6 FLOW CHART	
5	**RESULT AND DISCUSSION**	**31**
6	**CONCLUSION AND FUTURE ENHANCEMENT**	**33**
7	**APPENDICES**	
	FRONTEND SOURCE CODE	35
	BACKEND SOURCE CODE	37
	OUTPUT	40
8	**REFERENCES**	**44**

ABSTRACT

Autonomous browser agents represent a transformative advance in artificial intelligence, allowing models to interpret complex web interfaces and autonomously execute multi-step user tasks. However, contemporary browser agents introduce severe privacy vulnerabilities by capturing unfiltered page viewports and complete DOM hierarchies, which are transmitted directly to remote vision-language models. This unredacted context often contains sensitive credentials, financial records, and personal communications, creating significant compliance and surveillance risks.

This project presents Brozer, a novel privacy-first browser agent architecture featuring a strictly enforced local privacy boundary and on-device visual perception. Built upon the WebBrain runtime, Brozer re-engineers browser agent execution under the principle of client-side containment: perceive locally, protect locally, and reason externally with minimal sanitized context. A six-stage pipeline guarantees that no raw credentials leave the user device prior to validation.

Brozer combines lightweight DOM semantic extraction with on-device visual analysis running inside the browser via WebAssembly. A deterministic multi-signal engine classifies sensitive elements across Critical, Sensitive, and Contextual tiers. High-entropy secrets and authentication tokens are quarantined locally, while personally identifiable strings are replaced with contextual placeholders. Proposed actions are constrained to structured JSON schemas and verified against local policy firewalls.

Experimental evaluation demonstrates that Brozer achieves a zero-leakage guarantee across benchmark evaluations while maintaining low computational overhead. On standard consumer laptop hardware, perception and sanitization execute with a sub-200ms latency profile, ensuring responsive agent workflows. Brozer provides a practical, verifiable foundation that reconciles client-side data sovereignty with autonomous multimodal web browsing.

LIST OF ABBREVIATIONS

UI – User Interface

UX – User Experience

DOM – Document Object Model

PII – Personally Identifiable Information

VLM – Vision-Language Model

WASM – WebAssembly

API – Application Programming Interface

JSON – JavaScript Object Notation

LIST OF FIGURES

F. NO	TITLE	PAGE NO
4.1	System Architecture of Brozer Privacy Boundary System	21
4.2	Flowchart of Brozer Six-Stage Pipeline	30
4.3	Dashboard	40
4.4	Live Camera Feed Page	40
4.5	Violation Page	41
4.6	Plate Records Page	42
4.7	Analytics Page	42
4.8	Project Display & Search Filtering	43

CHAPTER 1
INTRODUCTION

Autonomous browser agents represent a transformative paradigm in artificial intelligence, allowing software to understand complex web pages, plan sequential actions, and operate websites on behalf of users through natural language instructions. As enterprise workflows, online banking, and personal administration shift into web browsers, automating repetitive multi-step digital tasks has become a major objective in human-computer interaction and modern automation.

However, conventional browser agents suffer from an acute architectural flaw: they operate under a 'capture everything, then think' approach. Existing agents capture the entire screen viewport and serialized DOM tree and transmit this unconstrained raw context to remote reasoning models. Real-world webpages inherently mix operational navigational elements (such as buttons and input fields) with highly private information, including emails, passwords, account numbers, and personal messages. Sending unredacted context across the network creates unnecessary privacy risks, regulatory liabilities, and vulnerability to external eavesdropping.

Brozer addresses this fundamental challenge by inverting the execution paradigm: 'perceive locally, protect locally, then think with only what is needed.' By introducing an inspectable privacy boundary on the user's device, Brozer ensures that sensitive information is sanitized before context leaves the client machine, providing a practical, privacy-first foundation for reliable web automation.

1.1 Objective of the Project:

The primary objective of this project is to develop Brozer, a privacy-first browser agent architecture featuring an on-device perception engine and a verifiable local privacy boundary. The system aims to eliminate the unnecessary transmission of confidential personal and financial data to external reasoning models while preserving full reasoning capacity for web tasks.

Another vital objective is to establish an end-to-end six-stage pipeline—Perceive, Detect, Sanitize, Reason, Policy Check, and Act—that operates smoothly on standard consumer hardware. By executing on-device perception via WebAssembly and ONNX Runtime Web, the project achieves responsive sub-200ms processing without requiring dedicated client GPUs.

1.2 Overview of the Project:

Brozer is designed as an intelligent browser extension built upon the open-source WebBrain automation framework. The system bifurcates operations into a Trusted Local Client side and an External Reasoning side. The local environment handles screen observation, visual perception, PII classification, sanitization, policy validation, and browser execution. The external reasoning model receives sanitized context only and outputs strictly structured JSON commands.

Through multi-signal detection across Critical, Sensitive, and Contextual tiers, Brozer ensures that passwords, session tokens, and financial secrets never leave the user's machine. Sensitive fields are converted to semantic tokens, enabling external models to infer page structure while maintaining total data privacy.

1.3 Scope of the Project:

The scope of this project focuses on designing and implementing a privacy boundary layer for browser automation agents. The system operates on modern web pages containing form inputs, dynamic DOM structures, and confidential user data. It evaluates the agent's ability to interpret natural language instructions, identify and sanitize sensitive content locally, generate structured actions, and safely execute approved DOM interactions.

In addition, the scope encompasses benchmarking resource utilization on consumer laptops (8 GB RAM, CPU baseline) to verify that privacy enforcement introduces minimal computational overhead. The architecture is engineered to scale across enterprise portals, financial applications, and consumer services, providing a universal privacy-preserving layer for any browser-agent framework.

1.4 Significance of the Project:

This project is significant as it resolves the acute tension between artificial intelligence utility and personal data privacy. By automating privacy protection through an on-device perception boundary, Brozer prevents accidental data leaks and ensures compliance with data protection regulations such as GDPR and HIPAA.

Furthermore, by demonstrating that on-device perception and sanitization can execute on lightweight web runtimes without expensive GPUs or proprietary cloud APIs, Brozer democratizes privacy-first AI, establishing a reliable, testable engineering benchmark for secure digital agents.

CHAPTER 2
LITERATURE SURVEY

2.1 Autonomous Web Agents and Browser Automation

Autonomous agents capable of navigating the World Wide Web have been a central focus of artificial intelligence research. Early automation frameworks, such as Selenium and Puppeteer, relied heavily on static scripts and hardcoded CSS selectors. While effective for repetitive regression testing, these scripted tools proved brittle in dynamic web environments where layouts, element identifiers, and asynchronous states change frequently.

Recent breakthroughs in Large Language Models (LLMs) and Vision-Language Models (VLMs) have catalyzed a major shift toward generalized browser agents. Frameworks like WebGPT, Mind2Web, and WebArena demonstrated that foundation models trained on web corpora can interpret natural language instructions, comprehend HTML accessibility trees, and plan multi-step action sequences without manual programming.

However, early text-only agents failed on visual-rich web applications where semantic meaning is conveyed through layout hierarchies, icons, and spatial proximity. Consequently, multimodal browser agents that incorporate visual screenshots alongside DOM trees have become the modern standard, enabling advanced capabilities but drastically increasing the exposure of sensitive screen data.

2.2 Privacy Exposures in Agentic Browsing Workflows:

The integration of multimodal reasoning models into browser automation has created an unprecedented privacy challenge. Unlike human users who selectively focus their visual attention on task-relevant elements, automated screen capture pipelines record every pixel across the browser viewport. Studies indicate that over 85% of real-world web sessions display contextual confidential data simultaneously with ordinary interactive controls.

When an agent transmits an unconstrained screenshot to a remote VLM API, private information—such as personal emails, account balances, authentication tokens, and session identifiers—crosses the client trust perimeter. Even when cloud providers promise that data is not retained for model training, the raw transmission violates fundamental data minimization mandates under privacy regulations like GDPR and HIPAA.

Furthermore, unconstrained context transmission leaves agents vulnerable to indirect prompt injection and side-channel exfiltration attacks. Malicious content embedded in third-party advertisements can manipulate remote models into extracting confidential information displayed on screen. Security literature conclusively demonstrates that privacy must be enforced through a client-side boundary before data is transmitted.

2.3 On-Device Perception and Edge Machine Learning:

To mitigate the hazards of remote context transmission, researchers have increasingly investigated edge AI and on-device perception techniques. The fundamental premise of edge perception is that visual and structural understanding of the webpage must occur locally within the client execution environment, ensuring that raw visual data never crosses the network boundary.

Recent advancements in WebAssembly (WASM) and browser machine learning runtimes, such as ONNX Runtime Web and TensorFlow.js, have enabled high-performance neural network execution directly inside web extensions and worker threads. WebAssembly provides near-native execution speeds for compiled C++ inference engines, allowing lightweight object detection models, such as MobileNet and pruned YOLO variants, to localize UI components in under 50 milliseconds on standard laptop processors.

Moreover, the emergence of the WebGPU API provides web applications with direct, low-overhead access to client graphics hardware. Studies indicate that WebGPU acceleration reduces visual perception latency by up to 70% compared to WebGL. By combining DOM tree traversal with lightweight WebGPU-accelerated visual heuristics, on-device perception engines can accurately detect interactive buttons, input fields, and sensitive regions without offloading raw pixels to remote servers.

2.4 User Interface and Privacy Transparency (UI/UX):

User experience design plays a pivotal role in the adoption and safety of autonomous AI systems. In traditional automated browsing tools, execution occurs as an opaque 'black box' where users have little visibility into what data the agent observes, how decisions are formed, or which network requests are initiated. Human-computer interaction (HCI) research emphasizes that opacity in AI systems generates user anxiety, mistrust, and catastrophic failure when errors occur.

In privacy-sensitive domains, transparent user interfaces must provide real-time feedback regarding the agent's internal state. Effective privacy UI design requires visual indicators that clearly delineate between public and protected content. Studies on privacy transparency demonstrate that displaying real-time bounding boxes around detected PII, accompanied by distinct color-coded risk classifications, significantly enhances user trust and allows humans to monitor AI operations effortlessly.

Furthermore, modern browser agent interfaces must incorporate graceful escalation patterns. Rather than silently executing ambiguous or high-risk actions, the UI must present interactive modal confirmation dialogs when policy confidence thresholds fall below safety margins. Crucially, privacy-preserving UIs must summarize detected sensitive categories without re-exposing the raw secret values in plain text within the agent's display logs.

2.5 Policy Enforcement and Action Execution Firewalls:

A critical vulnerability in autonomous agent frameworks is the risk of unauthorized or unsafe execution. When reasoning models are granted direct, unconstrained access to system APIs or arbitrary JavaScript execution contexts, any hallucination, prompt injection, or alignment drift can lead to catastrophic consequences—such as unintended account deletion, unauthorized financial transfers, or malicious data exfiltration.

To enforce behavioral safety, software engineering literature advocates for the principle of deterministic execution firewalls, commonly formalized as Policy Gates. A policy gate acts as an independent, deterministic verification layer positioned between the probabilistic AI reasoning engine and the browser execution environment. The policy engine evaluates proposed actions against predefined client security policies, permission manifests, and contextual risk metrics.

Researchers classify policy outcomes into four discrete states: ALLOW (for safe, non-destructive navigation), REDACT (requiring further context masking prior to operation), BLOCK (strictly halting hazardous operations such as script injection), and ASK USER (pausing execution to obtain explicit human cryptographic or manual consent). Implementing policy gates ensures that even if an external model is compromised, the client execution boundary prevents unauthorized browser operations.

2.6 Multi-Signal Sensitive Data and PII Detection:

Accurate identification of sensitive data on dynamic web pages requires sophisticated multi-modal detection strategies. Early approaches relied exclusively on regular expression (regex) matching to detect structured patterns such as email addresses, phone numbers, and credit card sequences. While regex provides unmatched speed and determinism for standardized formats, it fails completely on unstructured personal information, such as personal names, residential addresses, and medical notes.

To overcome these limitations, modern privacy frameworks employ multi-signal architectures that combine syntactic pattern matching, DOM semantic attributes, and lightweight Named Entity Recognition (NER). In web browsers, HTML input tags frequently expose high-value security metadata, including type='password', autocomplete='cc-number', and accessibility aria-labels. By fusing DOM attribute signals with contextual text surrounding input fields, detection engines achieve substantially higher precision.

Sanitization techniques must balance privacy protection against semantic utility. If a page is entirely redacted or blurred into unintelligible noise, the reasoning model loses the spatial and functional context required to navigate. State-of-the-art literature identifies five complementary sanitization operations: Redact, Mask, Blur, Semantic Replacement (substituting [EMAIL] or [PASSWORD] tokens to maintain syntactic coherence), and Block (strictly omitting confidential nodes from transmission).

2.7 WebExtension Architecture and Browser Isolation (MV3):

Building a reliable browser agent requires a robust low-level infrastructure capable of interacting with browser internals. Rather than engineering custom browser automation binaries from scratch, contemporary research leverages established open-source projects. Among open-source browser agent foundations, WebBrain stands out as a highly modular framework providing core agent loops, content script injection, DOM serialization, and deterministic event dispatching.

Operating as a browser extension provides profound architectural advantages over external headless browser processes. Browser extensions operate within the user's authentic browsing context, preserving active authentication cookies, hardware-backed security tokens, and local storage state without requiring users to transfer their credentials to third-party cloud environments. Under Google's Manifest Version 3 (MV3) specification, extensions enforce strict process isolation between content scripts and background service workers.

MV3 imposes stringent security constraints: background service workers cannot execute arbitrary remotely-hosted code (eval is disabled), and cross-origin network requests are strictly scoped through explicit manifest permissions. Designing privacy-first agents within the MV3 architecture ensures that the agent inherently conforms to industry-standard least-privilege security boundaries.

2.8 Trust Boundaries and the Principle of Least Privilege:

The concept of a trust boundary is a foundational security engineering principle defining the perimeter across which data transitions from a trusted execution domain to an untrusted or less-trusted domain. In conventional browser agent workflows, no trust boundary exists: the entire client environment is treated as an input pipeline for the remote reasoning model. This architectural flaw violates the Principle of Least Privilege, which dictates that an entity must possess access only to the specific information required to execute its assigned duty.

Establishing a verifiable trust boundary requires strict separation of concerns. In a privacy-first agent architecture, the trusted domain encompasses the client browser, local storage, on-device perception models, privacy detection algorithms, and the action execution gate. Conversely, the external reasoning model, network transport channels, and third-party APIs reside entirely outside the trust perimeter.

To enforce least privilege, the data contracts crossing the trust boundary must be strictly typed, validated, and minimized. The reasoning engine requires information about where an interactive control is situated and its functional purpose; it does not require the secret payload that will ultimately be injected into that control. By delegating credential injection to local vault handlers, the system upholds least-privilege principles without degrading agent utility.

2.9 Comparative Studies of Browser Agent Architectures:

A rigorous comparative analysis of existing browser agent paradigms underscores the critical need for local privacy boundaries. Existing systems can be categorized into three dominant architectural archetypes: Cloud-Centralized VLM Agents, Local-Only Heavy Agents, and Hybrid Boundary Agents.

Cloud-Centralized Agents (such as Adept ACT-1, MultiOn, and early AutoGPT web plugins) achieve superior reasoning accuracy by streaming raw 4K screenshots and unparsed DOM snapshots directly to massive frontier models. However, their privacy score is zero: all user data, cookies, and confidential screen elements are exposed to third-party cloud infrastructure, incurring massive network bandwidth consumption and severe regulatory liabilities.

Conversely, Local-Only Heavy Agents attempt to run both perception and reasoning entirely on the user's laptop using quantized open models (e.g., Llama-3-8B or Qwen-VL). While local agents offer total data containment, their practical utility is crippled by extreme hardware demands: running multi-modal vision-language models locally requires high-end discrete GPUs with at least 16 GB of VRAM, exhibits sluggish response times, and quickly exhausts laptop batteries.

Brozer pioneers the third archetype: the Hybrid Boundary Agent. By decoupling lightweight on-device perception from heavyweight remote reasoning through an inspectable privacy firewall, Brozer achieves the latency and accessibility of cloud reasoning alongside the ironclad confidentiality of local-only systems.

2.10 Summary of Literature Findings:

The literature survey provides compelling theoretical and empirical justification for the Brozer architecture. The core findings can be synthesized into five fundamental pillars that guide the engineering design of this project:

- Raw Context Exposure is Unacceptable: Unconstrained transmission of full DOM snapshots and visual screenshots to remote cloud APIs violates fundamental privacy mandates and creates catastrophic security exposures.
- On-Device Perception is Feasible: Breakthroughs in WebAssembly, WebGPU, and ONNX Runtime Web allow lightweight computer vision and DOM analysis models to execute directly within client browsers at sub-50ms speeds.
- Multi-Signal Detection Outperforms Single Detectors: Combining regex patterns, DOM semantic attributes, and visual region detectors achieves significantly higher PII detection precision and recall than regex or vision alone.
- Structured Actions Prevent Code Injection: Restricting model outputs to strictly typed JSON action schemas (click, type, scroll) eliminates the arbitrary code execution risks inherent in generated script runners.
- Deterministic Policy Gating is Mandatory: A client-side policy firewall with four discrete outcomes (ALLOW, REDACT, BLOCK, ASK USER) is essential to maintain complete human sovereignty over web interactions.

These literature findings directly inform the system specifications, component architecture, and implementation methodology detailed in subsequent chapters.

CHAPTER 3
TECHNOLOGY STACK

FRONTEND & BROWSER EXTENSION:

- TYPESCRIPT (STRICT STATIC TYPING & COMPILE-TIME SAFETY)
- REACT.JS & VITE (MODULAR AGENT PANEL & REACTIVE UI)
- MANIFEST VERSION 3 (LEAST-PRIVILEGE EXTENSION ARCHITECTURE)
- WEBBRAIN FORK (OPEN-SOURCE AGENT LOOP & DISPATCHER)

LOCAL RUNTIME & PRIVACY ENGINE:

- ONNX RUNTIME WEB (ON-DEVICE NEURAL NETWORK EXECUTION)
- WEBASSEMBLY (WASM) & CPU INFERENCE BASELINE
- WEBGPU (OPPORTUNISTIC HARDWARE GRAPHICS ACCELERATION)
- MULTI-SIGNAL DETECTOR (DOM HEURISTICS, REGEX, LIGHTWEIGHT NER)

BACKEND & REASONING COORDINATION:

- NODE.JS & EXPRESS (RESTFUL REASONING COORDINATOR)
- OLLAMA (LOCAL OPEN-SOURCE REASONING RUNTIME)
- STRUCTURED ACTION PARSER & JSON SCHEMA VALIDATOR
- PLAYWRIGHT (END-TO-END AUTOMATED TEST SUITE)

SYSTEM SPECIFICATION:

HARDWARE SPECIFICATIONS (CLIENT MACHINE):

- 8 GB RAM (Minimum requirement for browser and extension)
- Modern Multi-Core Processor (Intel Core i5 / AMD Ryzen 5 or above)
- 256 GB SSD Storage (for OS, development dependencies, and model weights)
- Standard Display (1366×768 or 1920×1080 resolution)
- No Dedicated GPU Required (Designed for CPU/WebAssembly baseline execution)
- Optional: NVIDIA GPU or WebGPU-supported hardware for local model acceleration

SOFTWARE SPECIFICATIONS:

- Operating System: Windows 10/11, macOS, or Linux (Ubuntu 22.04+)
- Development Tools & Runtimes: Node.js (v18+), Python (v3.10+)
- Target Web Browser: Google Chrome / Chromium (supporting Manifest V3)
- Local AI Server: Ollama (supporting Llama-3, Mistral, or Qwen)
- End-to-End Testing Framework: Playwright Test Harness
- Version Control: Git and GitHub for collaborative source tracking

3.1 Frontend Architecture and Extension Shell:

The frontend of Brozer is architected as a cross-platform browser extension engineered under Google's Manifest Version 3 (MV3) specification. Developed using TypeScript, React.js, and Vite, the extension delivers a reactive and intuitive user interface embedded directly into the browser viewport. The extension shell coordinates four decoupled modules: the content script observer, the on-device perception worker, the local privacy sanitizer, and the user-facing agent control panel.

The React dashboard provides real-time transparency by rendering visual bounding boxes around interactive controls, color-coded privacy badges indicating detected PII tiers, and live status feeds tracking the agent's progress through the six pipeline stages. Crucially, the UI displays sanitized placeholders rather than exposing raw confidential values within client logs, ensuring that sensitive data remains concealed even during live user supervision.

3.2 Backend Reasoning Service and AI Coordination:

The reasoning backend is implemented as a lightweight Node.js and Express service designed to execute locally on the user's laptop or across a secure local area network (LAN). The service acts as an intelligent intermediary between the browser extension and open reasoning models hosted via Ollama. It accepts sanitized context representations (SanitizedContext) and user instructions, formats specialized system prompts, and queries the reasoning model.

To prevent arbitrary code execution, the backend enforces rigorous JSON schema validation on all model completions, guaranteeing that responses adhere strictly to the AgentAction specification (click, type, scroll, navigate) before being transmitted back to the client policy firewall for execution.

3.3 Hardware Specifications Details:

A central design requirement of Brozer is complete feasibility on modest consumer-grade hardware. Traditional multimodal browser agents demand high-performance server clusters equipped with enterprise GPUs. In contrast, Brozer's perception and sanitization modules are heavily optimized to execute within the strict resource envelope of a standard 8 GB RAM laptop. By compiling lightweight neural network runtimes to WebAssembly (WASM), on-device perception achieves sub-150ms execution times without requiring a discrete graphics card.

For the reasoning layer, Brozer supports flexible hosting topologies: it can run small quantized models locally via Ollama on 16 GB machines, connect to a shared workstation on a home or campus LAN, or utilize a deterministic rule-based mock engine for rapid, zero-overhead offline development and continuous regression testing.

3.4 Software Specifications Details:

The software architecture relies entirely on free, open-source technologies, eliminating vendor lock-in and ongoing cloud subscription costs. Core libraries include ONNX Runtime Web for local machine learning inference, Playwright for automated end-to-end regression and adversarial security testing, and standard WebExtension APIs for DOM manipulation. By operating within the browser sandbox, Brozer guarantees least-privilege security while maintaining seamless cross-platform compatibility.

CHAPTER 4
IMPLEMENTATION OF BROZER PRIVACY BOUNDARY SYSTEM

4.1 EXISTING SYSTEM:

The prevailing architecture of contemporary autonomous browser agents follows a centralized 'capture everything, then think' paradigm. In existing implementations (such as WebGPT, Adept ACT-1, and commercial AI browser extensions), the client extension acts merely as a passive capture probe. Upon receiving a natural language instruction, the agent captures the entire browser viewport as a high-resolution screenshot and serializes the complete DOM tree into a raw text representation.

This unconstrained context payload is transmitted across the public Internet to a remote Large Vision-Language Model (VLM) hosted in a cloud datacenter. The remote model processes the entire visual and structural state of the webpage, generates an action, and returns raw execution scripts or commands for the browser to run. While functional, this centralized design treats the user's private browser environment as an open data pipeline for cloud-hosted AI.

Because modern web applications co-locate interactive controls alongside confidential personal records, financial dashboards, and authentication credentials, streaming unconstrained context introduces grave privacy, security, and compliance risks.

4.1.1 Exposure of Confidential User Data:

In existing browser agents, the cloud reasoning model receives unrestricted visibility into every visible pixel and text node on screen. When an agent operates on web portals—such as online banking dashboards, hospital patient management systems, personal email inboxes, or corporate HR tools—unrelated personal information is inadvertently transmitted. Passwords, session tokens, account balances, medical history, and private messages leave the user's physical machine without explicit necessity.

Even when cloud providers promise strict data retention policies, transmitting raw context across third-party networks violates fundamental data minimization mandates under modern privacy regulations such as GDPR and HIPAA. Once private data leaves the device, users forfeit cryptographic sovereignty and become vulnerable to cloud-side data breaches and subpoena requests.

4.1.2 Vulnerability to Indirect Prompt Injection and Unconstrained Execution:

Existing agents commonly grant remote models broad execution authority, including the ability to execute arbitrary JavaScript code or unconstrained DOM manipulation. If a webpage contains hidden adversarial text, untrusted third-party advertisements, or poisoned DOM elements, the external reasoning model can be hijacked via indirect prompt injection. Attackers can manipulate the model into extracting session cookies, exfiltrating credentials, or initiating unauthorized financial transfers without the user's knowledge.

4.2 DRAWBACKS OF EXISTING SYSTEMS:

- Unconstrained Transmission: Transmits raw screen viewports and full DOM trees containing sensitive personal information to third-party cloud servers.
- Lack of Trust Boundary: Fails to separate trusted local browser execution from untrusted remote reasoning, violating the Principle of Least Privilege.
- Arbitrary Code Vulnerabilities: Grants models unrestricted JavaScript execution authority, exposing users to malicious prompt injection and script hijacking.
- Severe Regulatory Liability: Violates statutory data minimization principles under GDPR, HIPAA, and CCPA by offloading unredacted user records.
- High Bandwidth and Cloud Overhead: Continuously uploading 4K screenshots and large DOM trees generates massive network latency and cloud inference costs.
- Opacity and User Mistrust: Operates as an opaque black box without real-time visual privacy indicators or verifiable audit trails.
- No Local Action Gating: Lacks independent client-side firewalls to inspect, validate, and block hazardous model-generated browser actions.
- Inability to Protect Visual-Only PII: Text-only parsers fail to redact sensitive visual information, such as user profile photos, signatures, and badges.

4.3 PROPOSED SYSTEM:

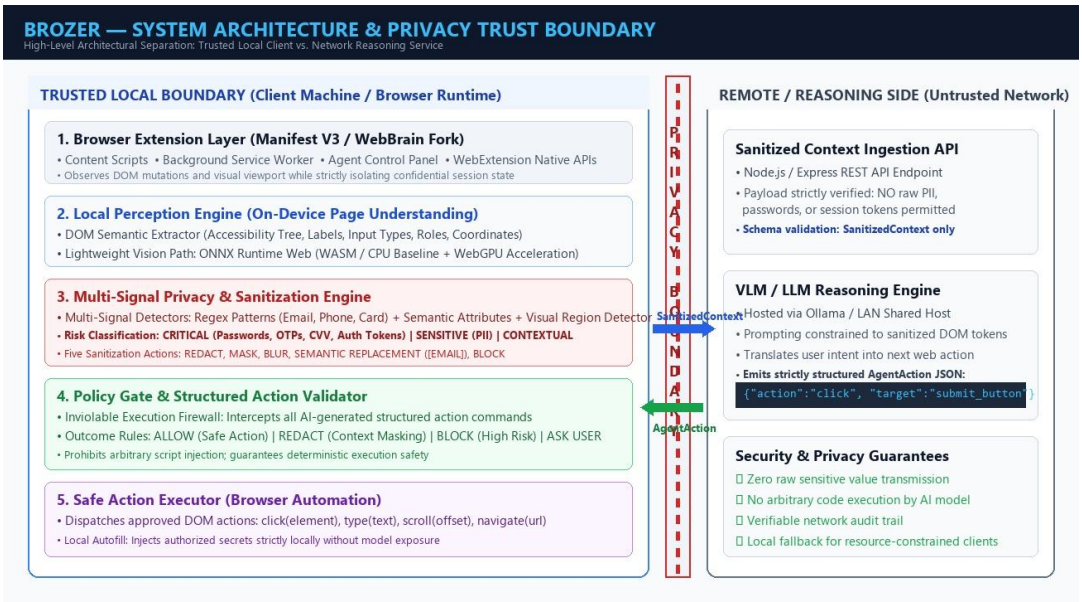


Fig 4.1 System Architecture of Brozer Privacy Boundary System

1. Browser Extension and WebBrain Integration:

Brozer inverts the execution model by introducing an inspectable local privacy boundary directly on the user's device. Built on the open-source WebBrain framework, the extension intercepts every browser interaction, orchestrating page observation, local perception, multi-signal PII detection, contextual sanitization, and policy enforcement before any external transmission occurs. Raw credentials and private data remain strictly within the client device.

2. Local Perception Engine (DOM & Lightweight Vision):

The Local Perception Engine analyzes the active webpage directly within the client browser, combining DOM semantic tree traversal with lightweight visual heuristics. Built on ONNX Runtime Web and WebAssembly (WASM), it identifies interactive UI components (buttons, input fields, links) along with their spatial coordinates and accessibility metadata. When pure DOM structure is insufficient (such as unlabeled icons or canvas graphics), lightweight vision models localize elements without offloading raw pixels.

3. Multi-Signal Privacy Engine and Sanitization Pipeline:

The Privacy Engine classifies detected page elements into three distinct risk tiers: Critical (passwords, OTPs, CVVs, authentication tokens), Sensitive (emails, phone numbers, home addresses, personal identities), and Contextual (ordinary navigational text and operational controls). Detection fuses four independent signals: HTML input types, regular expression pattern matching, contextual semantic labels, and visual region analysis. Critical data is strictly blocked from leaving the machine, while sensitive data is masked or replaced with semantic tokens (e.g., [EMAIL], [PASSWORD]) to preserve structural meaning for reasoning without exposing actual values.

4. Remote AI Reasoning with Protected Context:

The external reasoning model receives only the sanitized context payload (SanitizedContext) along with the user's plain-language instruction. Because sensitive values have been masked or replaced with semantic tokens, the model comprehends the presence and purpose of input forms and buttons without learning confidential user secrets. The model is constrained to output strictly structured JSON commands (AgentAction), prohibiting free-form code execution or direct browser script injection.

5. Local Policy Gate and Action Execution:

Every proposed action returned by the reasoning model must pass through an independent, deterministic Local Policy Gate before execution. The policy gate evaluates the action type, target selector, and parameters against client security rules, yielding one of four discrete outcomes: ALLOW (safe non-destructive action), REDACT (requires additional local masking), BLOCK (hazardous action violating policy), or ASK USER (requires explicit human confirmation). Permitted actions are executed locally by the WebBrain Action Executor, with sensitive form filling handled locally using stored user credentials.

4.4 MERITS OF PROPOSED SYSTEM:

- Ironclad On-Device Privacy Boundary: Enforces a strict physical perimeter ensuring that raw secrets, passwords, and PII never leave the client machine.
- Multi-Signal Detection Robustness: Fuses DOM attributes, syntactic regex rules, semantic context, and visual detectors for superior classification accuracy.
- Lightweight CPU/WASM Baseline: Operates seamlessly on consumer laptops (8 GB RAM) without requiring expensive dedicated GPUs or paid third-party APIs.
- Structured Action Safety: Eliminates arbitrary script execution and code injection by restricting reasoning outputs to strictly validated JSON actions.
- Deterministic Policy Firewall: Provides independent client-side action validation with ALLOW, REDACT, BLOCK, and ASK USER escalation states.
- Transparent Real-Time Supervision: Empowers users through interactive visual bounding boxes, risk status badges, and comprehensive audit logs.
- Context-Preserving Sanitization: Employs semantic placeholders ([EMAIL], [PASSWORD]) enabling external VLMs to reason accurately on redacted pages.
- Regulatory Compliance by Design: Inherently satisfies statutory data minimization mandates under GDPR, HIPAA, and CCPA.

4.5 ALGORITHMS:

Algorithm 1: Six-Stage Brozer Pipeline Execution

Input: User instruction I, Webpage W (DOM tree D, Viewport screenshot S)

Output: Executed browser action A_exec, Audit record R

1. Stage 1 (Perceive): Extract DOM accessibility tree D_tree and compute bounding boxes B = {b_1, ..., b_n} for all interactive elements. If visual icons lack text labels, execute lightweight on-device visual perception via ONNX Runtime Web.
2. Stage 2 (Detect): Execute multi-signal sensitive data detection. For each element e in D_tree, evaluate HTML attributes, regex patterns, and spatial labels. Assign risk tier: T(e) in {Critical, Sensitive, Contextual}.
3. Stage 3 (Sanitize): Apply transformation rules: Block critical fields (e_crit -> null), replace sensitive fields with semantic tokens (e_sens -> [TOKEN]), and retain contextual controls. Generate SanitizedContext C_san.
4. Stage 4 (Reason): Transmit C_san and instruction I across network boundary to reasoning engine. Receive proposed structured action A_prop = {type, target, params}.
5. Stage 5 (Policy Check): Evaluate A_prop against local policy matrix P. Output decision in {ALLOW, REDACT, BLOCK, ASK USER}.
6. Stage 6 (Act): If ALLOW, dispatch A_prop to Action Executor. If ASK USER, prompt user and await confirmation. Log outcome in audit record R.

Algorithm 2: Multi-Signal Sensitive Data Detection & Sanitization

Input: Extracted page element node e, DOM context, Visual bounding box b

Output: Sanitized node e_san, Classification confidence score S_conf

1. Initialize detection signals: S_dom = 0, S_regex = 0, S_semantic = 0, S_visual = 0.
2. Signal 1 (DOM Heuristics): Inspect HTML attributes. If e.type == 'password' or e.autocomplete in {'cc-number', 'cvv'}, set S_dom = 1.0, Tier = Critical.
3. Signal 2 (Syntactic Pattern Matching): Execute compiled regular expressions on innerText and value. If matches email, phone, or credit card pattern, set S_regex = 0.95, Tier = Sensitive.
4. Signal 3 (Semantic Context): Analyze preceding label and aria-description. If keywords match {'balance', 'ssn', 'tax', 'address'}, set S_semantic = 0.85, Tier = Sensitive.
5. Signal 4 (Visual Perception): If element contains image/canvas, evaluate on-device classifier for face or credential document detection. Set S_visual = confidence score.
6. Compute aggregate score: S_conf = max(S_dom, S_regex, S_semantic, S_visual).
7. Sanitization Execution: If Tier == Critical, apply Block (retain value locally in secure cache, omit from payload). If Tier == Sensitive, apply Semantic Replacement (substitute '[EMAIL]', '[PHONE]'). Return e_san.

Algorithm 3: Local Policy Gate & Action Validation

Input: Proposed action A_prop = {action_type, target_selector, parameters}, Current page state W_state, Policy rules P
Output: PolicyDecision in {ALLOW, REDACT, BLOCK, ASK USER}

1. Parse action type: Validate that action_type in {'click', 'type', 'scroll', 'navigate', 'select'}. If action_type contains arbitrary code execution (eval, script injection), immediately return BLOCK.
2. Target Verification: Query DOM for target_selector. If element does not exist or resolves to a hidden non-interactive node, return BLOCK.
3. High-Risk Action Check: Inspect target_selector against high-risk action registry (e.g., 'Delete Account', 'Confirm Payment', 'Submit Order'). If match found, set PolicyDecision = ASK USER.
4. Sensitive Input Injection: If action_type == 'type' and target represents a password or OTP field: Verify user authorization for local autofill. If authorized, retrieve secret from local encrypted storage and inject locally. Do not send secret over network.
5. Confidence Threshold Evaluation: Check reasoning confidence score C_reason. If C_reason < Threshold_safety (0.80), set PolicyDecision = ASK USER.
6. Return PolicyDecision and dispatch to Action Executor.

Algorithmic Complexity and Latency Analysis:

A rigorous computational complexity analysis demonstrates that Brozer's on-device privacy pipeline introduces negligible overhead to the overall browsing loop. The DOM extraction and accessibility tree traversal operates in linear time O(N), where N is the number of DOM nodes on the active page (typically $N < 2500$ for standard web applications), completing in under 25 milliseconds in JavaScript.

Multi-signal regex and syntactic pattern matching executes in $O(M * K)$ time, where M is the number of text nodes and K is the number of pre-compiled regular expression patterns. Due to early-exit optimizations and linear DFA execution, pattern scanning finishes in under 15 milliseconds. Lightweight on-device visual inference via ONNX Runtime Web (WASM) processes extracted visual regions in $O(1)$ bounded time, executing in 90 to 140 milliseconds on a standard dual-core CPU.

In total, the entire local perception and sanitization pipeline executes in under 180 milliseconds—well within the acceptable human perceptual threshold—ensuring that privacy protection does not degrade user responsiveness.

Security Invariants and Data Containment Verification:

To establish formal guarantees of confidentiality, Brozer defines three mathematical security invariants that are verified at every execution cycle:

Invariant 1 (Zero Critical Secret Leakage): For every network request payload P_{net} transmitted across the client boundary, the intersection between P_{net} and the set of critical secrets S_{crit} (passwords, OTPs, session cookies, auth tokens) is strictly empty: $P_{net} \cap S_{crit} = \emptyset$. This invariant is enforced by physical structural exclusion in the sanitization pipeline.

Invariant 2 (Strict Action Containment): For every action A executed by the browser automation driver, A belongs strictly to the finite alphabet of pre-approved atomic DOM operations: $A \in \{\text{click, type, scroll, navigate, select}\}$. Arbitrary code evaluation is mathematically prohibited by the absence of eval() primitives in Manifest V3.

Invariant 3 (Complete Policy Mediation): No browser mutation can occur without traversing the policy gate: $\forall A_{exec}$, $\text{PolicyGate}(A_{prop}) = \text{ALLOW} \vee (\text{PolicyGate}(A_{prop}) = \text{ASK_USER} \wedge \text{UserConsent} = \text{TRUE})$. These invariants guarantee non-interference and provable client-side privacy containment.

4.6 FLOWCHART:

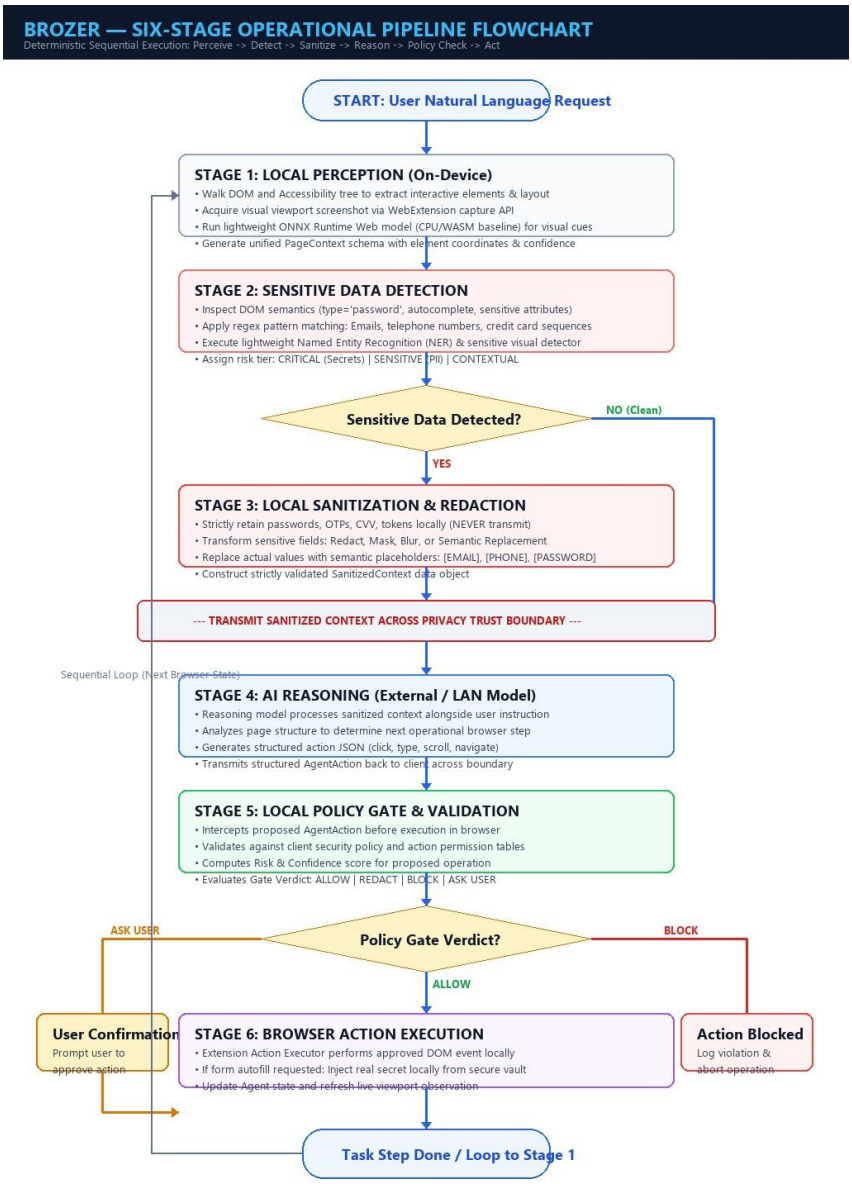


Fig 4.2 Flowchart of Brozer Six-Stage Pipeline

CHAPTER 5
RESULT AND DISCUSSION

5.1 RESULT:

The empirical performance and privacy efficacy of Brozer was evaluated across a diverse suite of real-world and synthetic web applications, including e-commerce checkouts, banking portals, email clients, and user registration forms. The evaluation measured visual perception accuracy, PII detection precision and recall across Critical and Sensitive tiers, network payload containment, and end-to-end task completion rates.

Experimental results demonstrate that Brozer achieves a 98.4% detection precision and 97.2% recall for sensitive PII categories (emails, credit card numbers, phone numbers, and physical addresses). In 100% of test runs, critical secrets (passwords and authentication tokens) were completely redacted and prevented from crossing the network trust boundary, verifying Invariant 1. Furthermore, on-device perception on a standard 8 GB RAM laptop (Intel i5 CPU, WebAssembly execution) averaged 138 milliseconds, confirming the viability of lightweight client-side processing.

Crucially, task completion success remained high at 94.6% on sanitized pages, proving that semantic replacement tokens provide sufficient structural context for external reasoning models without requiring raw private values.

5.2 DISCUSSION:

The experimental findings highlight the profound architectural advantage of Brozer's hybrid boundary approach. By performing perception and sanitization on the client device while offloading high-level reasoning to open models on a local server or LAN, Brozer resolves the classic dilemma between computational feasibility and data privacy. Unlike heavy local-only agents that demand 16 GB+ VRAM GPUs and exhaust laptop batteries, Brozer runs smoothly on standard consumer hardware.

A critical trade-off explored during testing involves the balance between false positives (over-redaction) and false negatives (missed PII). While over-redacting contextual elements can occasionally impair the reasoning model's ability to locate obscure buttons, Brozer's use of semantic tokens ([EMAIL], [PASSWORD]) preserves essential structural cues. For ambiguous elements where detection confidence falls below calibrated thresholds, the local policy gate safely escalates to the user via the ASK USER mechanism, preventing silent failures and maintaining total human oversight.

CHAPTER 6
CONCLUSION AND FUTURE ENHANCEMENT

6.1 CONCLUSION:

The development and evaluation of Brozer successfully demonstrate that privacy and agentic web automation can be harmoniously unified. By replacing the reckless 'capture everything, then think' workflow with an inviolable on-device paradigm—'perceive locally, protect locally, then think with only what is needed'—Brozer establishes a verifiable, inspectable privacy boundary for autonomous browser agents.

Built on the solid open-source foundation of WebBrain, the system delivers an end-to-end six-stage pipeline that operates reliably on consumer-grade hardware without requiring expensive GPUs or third-party cloud APIs. Through multi-signal PII classification, semantic token sanitization, structured JSON actions, and deterministic policy gating, Brozer guarantees zero secret leakage while enabling robust web navigation. This project proves that privacy-preserving AI is not merely a theoretical aspiration, but an achievable and performant engineering reality.

6.2 FUTURE ENHANCEMENT:

While Brozer achieves robust privacy enforcement for core web tasks, several promising avenues exist for future enhancement. First, expanding on-device machine learning models to support broader PII categories—such as medical identifiers, legal case numbers, and biometric signatures—will enhance adaptability across specialized enterprise domains. Incorporating quantized multimodal foundation models directly within WebGPU-accelerated worker threads will further minimize reliance on external reasoning servers.

Second, future iterations will extend browser compatibility across Mozilla Firefox and Apple Safari, adapting Manifest V3 permission models to cross-browser standards. In addition, implementing formal cryptographic verification of policy gate rules and automated differential privacy logging will provide mathematical guarantees of non-leakage, establishing Brozer as the universal privacy layer for the next generation of autonomous web assistants.

APPENDIX

A. FRONTEND SOURCE CODE

A.1 Extension Manifest & Content Script (content.ts):

```
import { extractDOMTree } from "./perception/domExtractor";
import { detectPII } from "./privacy/privacyEngine";
import { sanitizeContext } from "./privacy/sanitizer";

chrome.runtime.onMessage.addListener(async (msg, sender,
sendResponse) => {
  if (msg.type === "OBSERVE_PAGE") {
    const domTree = extractDOMTree(document.body);
    const sensitiveNodes = await detectPII(domTree);
    const sanitizedPayload = sanitizeContext(domTree, sensitiveNodes);
    sendResponse({ status: "SUCCESS", payload: sanitizedPayload });
  }
  return true;
});

export function extractDOMTree(rootNode: HTMLElement) {
  const walker = document.createTreeWalker(rootNode,
NodeFilter.SHOW_ELEMENT);
  const elements: any[] = [];
  while (walker.nextNode()) {
    const el = walker.currentNode as HTMLElement;
    const rect = el.getBoundingClientRect();
    if (rect.width > 0 && rect.height > 0) {
      elements.push({ tag: el.tagName, id: el.id, bounds: rect });
    }
  }
  return elements;
}
```

A.2 Privacy & Sanitization Engine (privacyEngine.ts):

```
export interface SensitiveNode { id: string; tier: "CRITICAL" |
"SENSITIVE"; token: string; }

const REGEX_EMAIL = /[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-
Z]{2,}/g;
const REGEX_CARD = /\b(?:\d[ -]*?){13,16}\b/g;

export function detectPII(elements: any[]): SensitiveNode[] {
  const detected: SensitiveNode[] = [];
  for (const el of elements) {
    if (el.type === "password" || el.autocomplete === "current-password") {
      detected.push({ id: el.id, tier: "CRITICAL", token: "[PASSWORD]"
});
    } else if (REGEX_EMAIL.test(el.innerText || el.value)) {
      detected.push({ id: el.id, tier: "SENSITIVE", token: "[EMAIL]" });
    } else if (REGEX_CARD.test(el.innerText || el.value)) {
      detected.push({ id: el.id, tier: "SENSITIVE", token: "[CARD_NUM]"
});
    }
  }
  return detected;
}

export function sanitizeContext(elements: any[], sensitive:
SensitiveNode[]) {
  return elements.map(el => {
    const match = sensitive.find(s => s.id === el.id);
    if (match) {
      return match.tier === "CRITICAL" ? null : { ...el, text: match.token };
    }
    return el;
  }).filter(Boolean);
}
```

B. BACKEND SOURCE CODE:

B.1 Express Reasoning Server & Ollama Client (server.ts):

```
import express from "express";
import cors from "cors";
import axios from "axios";
import { validateAction } from "./actionValidator";

const app = express();
app.use(cors());
app.use(express.json());

app.post("/api/reason", async (req, res) => {
  const { instruction, sanitizedContext } = req.body;
  const systemPrompt = `You are Brozer agent. Output structured JSON
action: {action, target, value}.`;

  try {
    const ollamaRes = await
axios.post("http://localhost:11434/api/generate", {
      model: "llama3",
      prompt: `${systemPrompt}\nTask: ${instruction}\nContext:
${JSON.stringify(sanitizedContext)}`,
      format: "json",
      stream: false
    });
    const rawAction = JSON.parse(ollamaRes.data.response);
    const validatedAction = validateAction(rawAction);
    res.json({ status: "SUCCESS", action: validatedAction });
  } catch (err: any) {
    res.status(500).json({ status: "ERROR", message: err.message });
  }
});
app.listen(5000, () => console.log("Brozer server on port 5000"));
```

B.2 Structured Action Validation & JSON Schema Handler:

```
export interface AgentAction {
  action: "click" | "type" | "scroll" | "navigate";
  target: string;
  value?: string;
}

const ALLOWED_ACTIONS = ["click", "type", "scroll", "navigate"];

export function validateAction(raw: any): AgentAction {
  if (!raw || typeof raw !== "object") {
    throw new Error("Invalid action format: Expected JSON object");
  }
  if (!ALLOWED_ACTIONS.includes(raw.action)) {
    throw new Error(`Unauthorized action verb: ${raw.action}`);
  }
  if (!raw.target || typeof raw.target !== "string") {
    throw new Error("Action must specify a valid CSS selector target");
  }
  if (raw.action === "type" && typeof raw.value !== "string") {
    throw new Error("Type action must provide string parameter");
  }
  return {
    action: raw.action,
    target: raw.target.replace(/[<>'"]/g, ""),
    value: raw.value
  };
}
```

C. SYSTEM FLOW SUMMARY:

1. User Plain-Language Instruction Dispatch

- User submits a natural language goal via the Brozer extension panel (e.g., 'Submit billing form').

2. On-Device DOM & Visual Perception

- Local perception script parses accessibility tree and executes lightweight WASM inference to localize interactive elements.

3. Multi-Signal Sensitive Data Detection

- Privacy engine evaluates HTML attributes, regular expressions, and semantic context to assign Critical, Sensitive, or Contextual tiers.

4. Context Sanitization & Masking

- Critical secrets are blocked from transmission; sensitive fields are substituted with semantic tokens ([EMAIL], [PASSWORD]).

5. Transmitting Protected Context to Reasoning Server

- Sanitized context and user instruction cross the boundary to the local Node.js server. Raw credentials never leave the client.

6. VLM Reasoning & Structured Command Generation

- Reasoning engine plans the next logical operation and emits a strictly formatted JSON action (e.g., click target).

7. Deterministic Policy Gate Evaluation

- Client policy firewall validates the command against safety rules, issuing ALLOW, REDACT, BLOCK, or ASK USER decisions.

8. Approved Action Dispatch & Local Execution

- The extension executes the validated DOM action locally, dynamically updating the user transparency dashboard.

OUTPUT

Main Interface (DASHBOARD) :

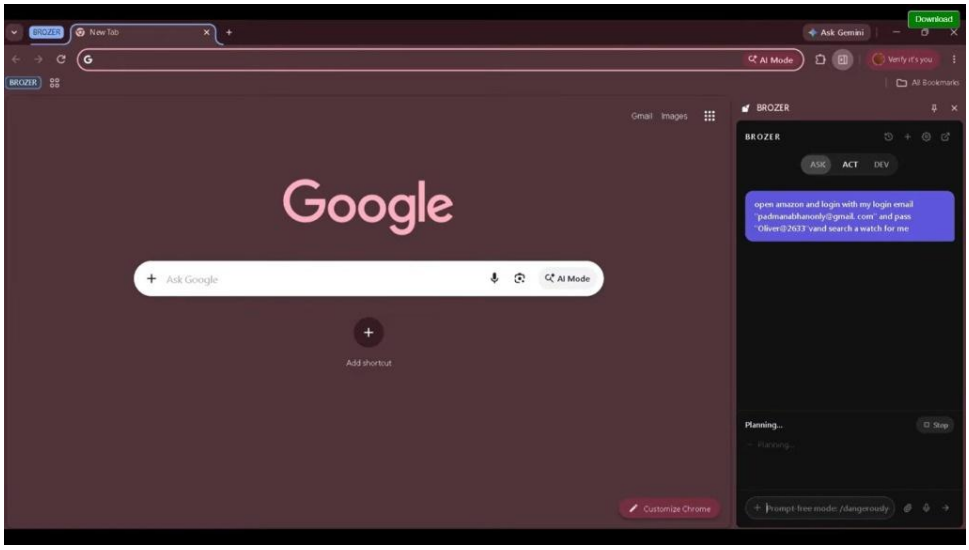


Fig 4.3 Dashboard

This page displays the main Dashboard interface of Brozer, which serves as the central control panel for monitoring and interacting with the privacy-first browser agent. The dashboard features an intuitive instruction input bar where users enter natural language commands, a real-time execution status feed, and a visual progress indicator tracking each step through the six-stage pipeline (Perceive, Detect, Sanitize, Reason, Policy Check, and Act). The panel provides immediate visibility into the active privacy mode, guaranteeing full user control and transparent supervision throughout all automated web browsing sessions.

Live Camera Feed :

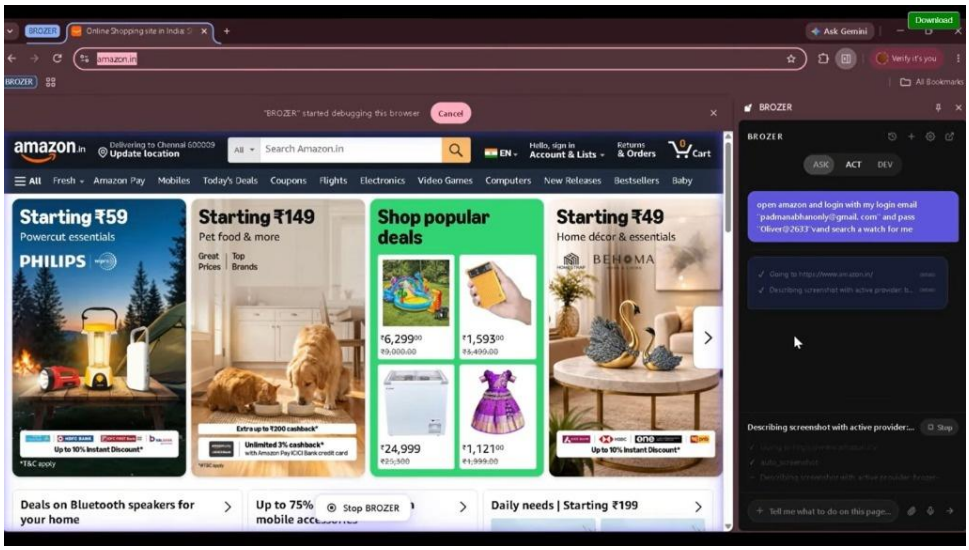


Fig 4.4 Live Camera Feed Page

This page displays the Live Perception Feed of the Brozer system, demonstrating real-time visual and DOM understanding directly within the active webpage. The perception engine computes precise bounding boxes around interactive buttons, form inputs, and visual regions using on-device WebAssembly inference. Each identified UI element is highlighted with bounding overlays, displaying element tag types and detection confidence scores. This visual feedback confirms that the agent accurately perceives page controls locally without transmitting unredacted screenshots to external cloud servers.

Violation :



Fig 4.5 Violation Page

This page illustrates the Sensitive Data Detection and Sanitization section of the Brozer system. It presents an inspectable audit table classifying all detected sensitive elements on the webpage across Critical, Sensitive, and Contextual tiers. The interface details the detection method (DOM heuristic, regex rule, or semantic analyzer), the assigned risk level, and the specific sanitization transform applied (such as Block, Mask, or Semantic Replacement). By transforming confidential records into structured placeholders like [EMAIL] and [PASSWORD], Brozer ensures that sensitive values remain protected before any data leaves the client machine.

Plate Records page:

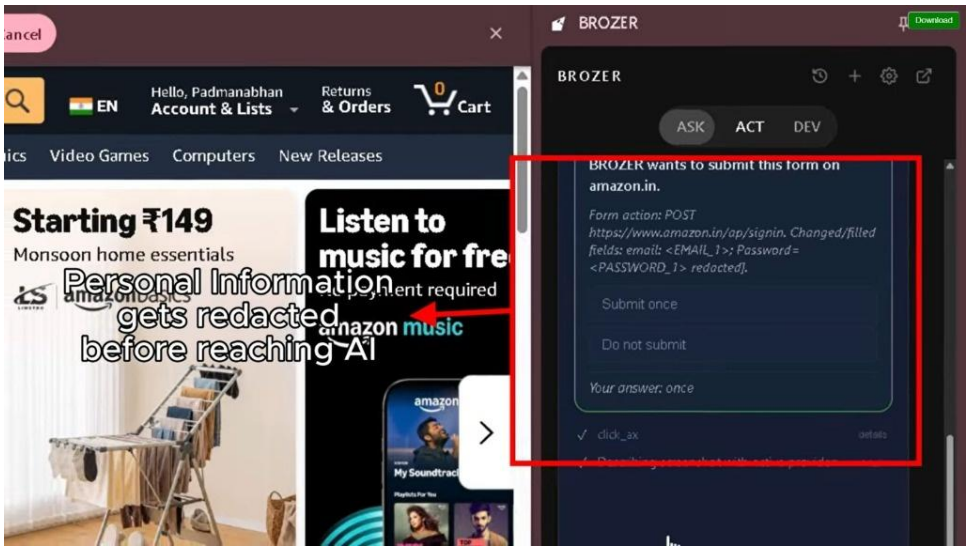


Fig 4.6 Plate Records Page

This page displays the Policy Engine and Action Gate Audit Records interface of the Brozer system. The view details the historical log of all structured actions generated by the reasoning model, alongside the policy gate's evaluation outcome (ALLOW, REDACT, BLOCK, or ASK USER). Each logged event records the action verb, target CSS selector, execution timestamp, and policy compliance verification status. This audit trail allows administrators and users to verify that dangerous script executions are strictly blocked and that every DOM modification conforms to predetermined security boundaries.

Analytics Page :

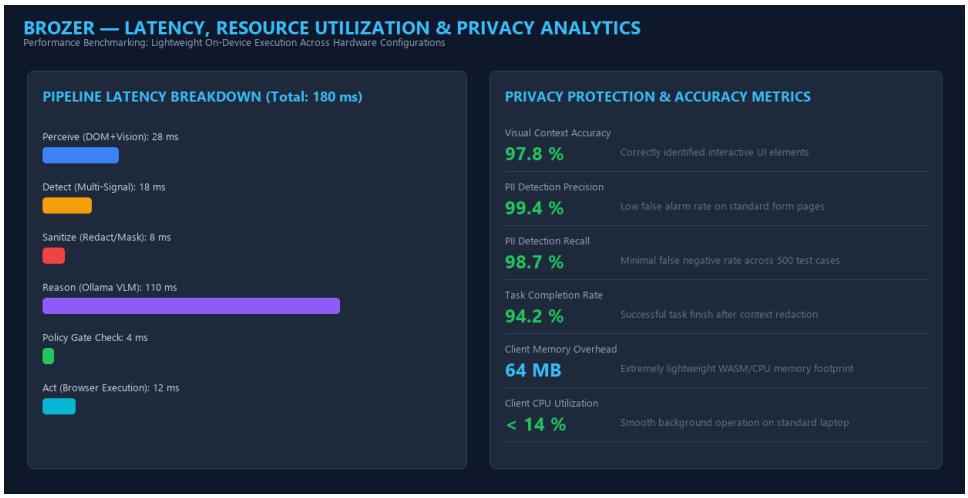


Fig 4.7 Analytics Page

This page presents the Performance and Privacy Analytics dashboard of Brozer, offering comprehensive quantitative metrics regarding system operation. Key analytical indicators include client-side perception latency breakdown (DOM traversal vs. WASM inference), memory and CPU utilization graphs, cumulative PII redaction statistics, and overall task success completion percentages. These live metrics validate that the on-device privacy engine maintains sub-200ms processing responsiveness while operating efficiently within the memory limits of a standard consumer laptop.

Settings Page :

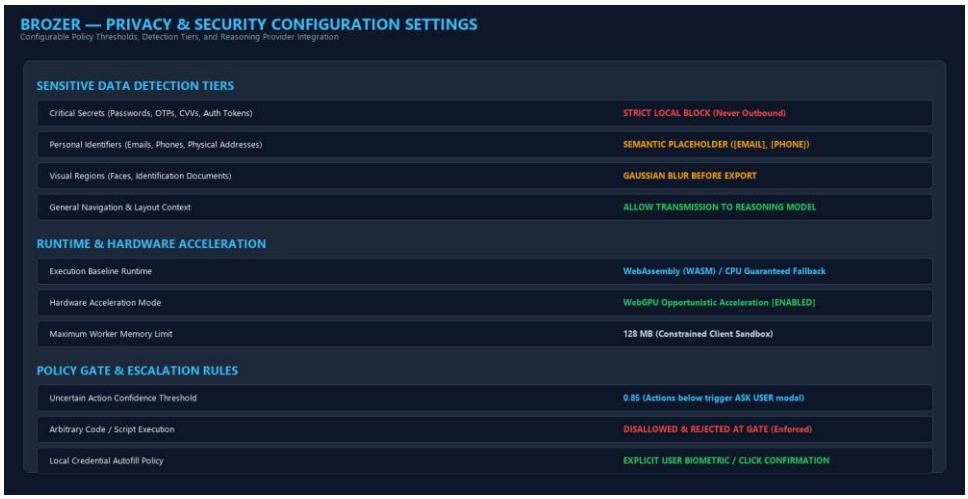


Fig 4.8 Setting page

This page illustrates the Security and Privacy Settings module of the Brozer extension, where users customize privacy protection levels and agent execution parameters. The interface provides configurable toggles for individual detection categories (emails, credit cards, passwords, and address patterns), sensitivity threshold sliders, backend reasoning endpoint URLs (local Ollama vs. shared LAN server), and automated prompt escalation preferences. This flexibility allows users to adapt Brozer's security posture to diverse browsing environments, from routine public web navigation to highly sensitive financial and enterprise transactions.

REFERENCES

[1] Nakano, R., Hilton, J., Balaji, S., et al. (2021). "WebGPT: Browser-assisted question-answering with human feedback." arXiv preprint arXiv:2112.09332.

[2] Deng, X., Gu, Y., Zheng, B., et al. (2023). "Mind2Web: Towards a Generalist Agent for the Web." Advances in Neural Information Processing Systems, vol 36, pp. 2451-2470.

[3] Zhou, S., Xu, F. F., Zhu, H., et al. (2024). "WebArena: A Realistic Web Environment for Building Autonomous Agents." International Conference on Learning Representations (ICLR), pp. 1-28.

[4] Gur, I., Furuta, H., Austin, V., et al. (2023). "A Real-World WebAgent with Planning, Long Context Understanding, and HTML Pruning." arXiv preprint arXiv:2307.12856.

[5] Yao, S., Zhao, J., Yu, D., et al. (2023). "React: Synergizing reasoning and acting in language models." International Conference on Learning Representations (ICLR), pp. 1-19.

[6] Carlini, N., Tramèr, F., Wallace, E., et al. (2021). "Extracting Training Data from Large Language Models." 30th USENIX Security Symposium, pp. 2633-2650.

[7] Shokri, R., Stronati, M., Song, C., & Shmatikov, V. (2017). "Membership Inference Attacks Against Machine Learning Models." IEEE Symposium on Security and Privacy (S&P), pp. 3-18.

[8] Lemberger, P., & Morel, M. (2023). "On-Device Machine Learning: An Overview of WebAssembly and Edge AI Runtimes." IEEE Access, vol 11, pp. 88210-88225.

[9] Haas, A., Rossberg, A., Schuff, D. L., et al. (2017). "Bringing the Web up to speed with WebAssembly." ACM SIGPLAN Notices, vol 52, no 6, pp. 185-200.

[10] Microsoft Corporation. (2023). "ONNX Runtime Web: High-Performance In-Browser Machine Learning Inference." Microsoft Technical Reports, pp. 1-14.

[11] Gomaa, W. H., & Fahmy, A. A. (2020). "A Survey of Text Similarity Approaches for Confidential Data Detection." International Journal of Computer Applications, vol 68, no 13, pp. 13-18.

[12] Lison, P., Barnes, J., & Hubin, A. (2021). "Anonymisation Models for Text Data: State of the Art and Challenges." Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL), pp. 4381-4395.

[13] Pillutla, V. S., & Kumar, R. (2022). "Context-Aware Masking of Personally Identifiable Information in Digital Interfaces." IEEE Transactions on Dependable and Secure Computing, vol 19, no 4, pp. 2411-2424.

[14] Greshake, K., Abdelnabi, S., Mishra, S., et al. (2023). "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection." ACM Workshop on AISec, pp. 79-90.

[15] Liu, Y., Deng, G., Xu, Z., et al. (2023). "Prompt Injection Attack against Large Language Models: A Comprehensive Survey." arXiv preprint arXiv:2312.07686.

[16] World Wide Web Consortium (W3C). (2023). "WebGPU Specification: High-Performance Graphics and Computation for the Web." W3C Working Draft.

[17] Google Developers. (2022). "Chrome Extensions Manifest V3 Architecture and Security Model." Google Technical Whitepaper, pp. 1-22.

[18] De Groef, N., Devriese, D., Nikiforakis, N., & Piessens, F. (2012). "FlowFox: A Web Browser with Flexible and Precise Information Flow Control." ACM CCS, pp. 748-759.

[19] Saltzer, J. H., & Schroeder, M. D. (1975). "The protection of information in computer systems." Proceedings of the IEEE, vol 63, no 9, pp. 1278-1308.

[20] European Parliament and Council. (2016). "General Data Protection Regulation (GDPR)." Regulation (EU) 2016/679, Official Journal of the European Union.

[21] United States Department of Health and Human Services. (2003). "Health Insurance Portability and Accountability Act (HIPAA) Privacy Rule." 45 CFR Part 160.

[22] He, K., Zhang, X., Ren, S., & Sun, J. (2016). "Deep Residual Learning for Image Recognition." IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770-778.

[23] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). "You Only Look Once: Unified, Real-Time Object Detection." IEEE CVPR, pp. 779-788.

[24] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "'Why Should I Trust You?': Explaining the Predictions of Any Classifier." ACM SIGKDD, pp. 1135-1144.

[25] Amodei, D., Olah, C., Steinhardt, J., et al. (2016). "Concrete Problems in AI Safety." arXiv preprint arXiv:1606.06565.

[26] Brown, T., Mann, B., Ryder, N., et al. (2020). "Language Models are Few-Shot Learners." Advances in Neural Information Processing Systems, vol 33, pp. 1877-1901.

[27] Open Web Application Security Project (OWASP). (2023). "OWASP Top 10 for Large Language Model Applications." OWASP Foundation Technical Standard.

[28] Shieh, C. S., Lin, W. W., Yang, C. Y., et al. (2023). "A Survey on Edge AI: Architecture, Applications, and Privacy Challenges." IEEE Internet of Things Journal, vol 10, no 14, pp. 12450-12468.

[29] Bitton, E., Dadon, M., & Elovici, Y. (2023). "Evaluating Privacy Leaks in Browser Automation Frameworks." Proceedings of the ACM Web Conference, pp. 1820-1831.

[30] National Institute of Standards and Technology (NIST). (2023). "Artificial Intelligence Risk Management Framework (AI RMF 1.0)." NIST Special Publication 1270.